

Monte Carlo Counterfactual Regret
Minimization
Independent Project Essay

Seongwoo Hur

Department of Statistics, University of Chicago

1 Monte Carlo Counterfactual Regret Minimization

For this project, I implemented Monte Carlo Counterfactual Regret Minimization (MCCFR), focusing on external sampling and outcome sampling. This project builds on my previous implementation and report on regret matching and full-tree CFR for Kuhn Poker [1]. The main question is what changes when CFR does not traverse the whole game tree, but instead estimates the regret update from sampled trajectories. I use Leduc Poker to compare full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR in Python.

1.1 From CFR to MCCFR

Counterfactual Regret Minimization (CFR) is a regret-minimization method for extensive-form games with imperfect information. The way I think about it is that every information set gets its own small regret minimizer. This is the same viewpoint I used in the earlier CFR project, based on Zinkevich et al.’s CFR paper and Farina’s regret-minimizer notes [4, 6, 7]. At each iteration, CFR computes a counterfactual value for every legal action, updates the local regrets, and then gets the next behavioral strategy by regret matching.

In the full-tree version of CFR, the update is exact for the current strategy profile because it traverses the entire game tree. This made the algorithm easy to check in my earlier Kuhn Poker implementation, but it is also the bottleneck: one iteration costs time proportional to the size of the tree. Kuhn Poker is tiny, so this is not an issue there. Leduc Poker is still small, but large enough that full tree traversal becomes noticeably more expensive. Once the game has more cards, betting actions, or public states, this full traversal cost becomes the main problem.

MCCFR keeps the same local regret-minimization structure but estimates the counterfactual update by sampling part of the tree. One version implemented here is *external sampling*. When updating player i , chance nodes and opponent nodes are sampled, while player i ’s own decision nodes still branch over all legal actions. So when the sampled path reaches an information set of player i , the algorithm can still compare all actions at that information set:

$$R_i^T(I, a) \leftarrow R_i^{T-1}(I, a) + \hat{v}_i(I, a) - \hat{v}_i(I),$$

where the hats indicate sampled estimates. This is the part that made external sampling easiest for me to connect back to vanilla CFR: the regret update has the same action-value minus node-value form, but the values come from a sampled subtree instead of the whole tree.

The other version is *outcome sampling*. It samples the update player’s action too, so one update follows essentially one terminal trajectory. Since the unchosen actions are not observed, the estimator needs importance weights based on the probability of the sampled trajectory. This also made the results noisier.

1.2 Leduc Poker as the Test Game

I used Leduc Hold’em as the main game for the experiments. It is a small poker game that is often used in CFR examples and opponent modeling work [3]. It is larger

than Kuhn Poker, but still small enough that I can compute exact best responses for evaluation. The deck contains six cards:

$$J, J, Q, Q, K, K.$$

Each player antes one chip and receives one private card. After the first betting round, one public card is revealed. A second betting round follows, and if no one folds, the hand goes to showdown.

Even though Leduc is small, it still has the main features I wanted:

- private information, because each player only observes their own card;
- sequential decisions, because betting history affects later actions;
- information sets, because the same private card and public history can arise under different hidden opponent cards;
- public chance, because the board card changes hand strength after the first betting round;
- bluffing, because weak hands can represent strong future or made hands.

The fixed-limit rules I used are the standard small Leduc rules. The round-1 bet size is 2, the round-2 bet size is 4, and each round has a maximum of two bet/raise actions. At showdown, a pair with the public card beats any high card. If neither player pairs the public card, the higher private rank wins, with $K > Q > J$. Equal private ranks split the pot. For the smaller sanity-check game, I used the standard Kuhn Poker rules from the earlier project [8, 2].

In the code, the game is represented as a simple dictionary of nodes. A chance node stores either the initial ordered private-card deal or the public-card deal. A decision node stores the acting player, the information set, and the legal actions. A terminal node stores the zero-sum utility vector. With the above parameters, the generated Leduc tree has 9,451 nodes and 288 information sets, compared with only a few dozen strategically relevant nodes in Kuhn Poker. This size was enough for the difference between full traversal and sampled traversal to show up in the experiment.

1.3 Regret Matching and Full-Tree CFR

Regret matching is the local learning rule I used in all three trainers. Suppose an information set has action set A . Let $R^T(a)$ be the cumulative regret for action a after T updates. Regret matching chooses the next mixed action by normalizing only the positive regrets:

$$\sigma^{T+1}(a) = \begin{cases} \frac{[R^T(a)]_+}{\sum_{b \in A} [R^T(b)]_+}, & \text{if } \sum_{b \in A} [R^T(b)]_+ > 0, \\ \frac{1}{|A|}, & \text{otherwise.} \end{cases}$$

The uniform fallback matters early in training because all regrets start at zero.

In full-tree CFR, each information set has its own regret table. At each iteration and for each update player i , my implementation recursively traverses the full game tree. Let I be an information set of player i , and let $a \in A(I)$. The update has the form

$$R_i^T(I, a) \leftarrow R_i^{T-1}(I, a) + \pi_{-i}^{\sigma^T}(I) \left(v_i(\sigma_{I \rightarrow a}^T, I) - v_i(\sigma^T, I) \right).$$

Here $\pi_{-i}^{\sigma^T}(I)$ is the counterfactual reach probability from chance and the opponent, $v_i(\sigma_{I \rightarrow a}^T, I)$ is the counterfactual value of deviating to action a at I , and $v_i(\sigma^T, I)$ is the value of following the current mixed action at I .

For the code, this can be written more concretely. At an update player node, recursively compute one child value for each legal action:

$$u(a) = \text{value returned by the child after action } a.$$

The value of the current mixed action is

$$u(\sigma) = \sum_{a \in A(I)} \sigma(I, a) u(a).$$

Then the regret increment is proportional to $u(a) - u(\sigma)$. The counterfactual reach factor is what makes the local regret update consistent with the global extensive-form game.

The full-tree implementation keeps two tables: cumulative regrets and cumulative average-strategy weights. The regret table determines the current strategy through regret matching. The strategy-sum table is used at the end to return the average strategy, which is the strategy that converges in the standard CFR guarantee.

1.4 External Sampling

I implemented external sampling with a recursive trainer. At a terminal node, the recursion returns the payoff of the update player. At a chance node, one chance outcome is sampled. At an opponent node, one opponent action is sampled from the opponent's current regret-matching strategy. At an update player node, every legal action is recursively evaluated, and the local regret increment is

$$\hat{v}(a) - \sum_{b \in A(I)} \sigma(I, b) \hat{v}(b).$$

This is the same action-value minus node-value form used in ordinary CFR.

The average strategy is stored in a tabular strategy-sum table. For external sampling, I used the simple sampled averaging rule: when an opponent information set is visited during a sampled traversal, the opponent's current strategy at that information set is added to the corresponding strategy sum. Since each iteration updates both players once, both players' average-strategy tables are updated over time from the sampled traversals.

1.5 Outcome Sampling and Importance Weights

Outcome sampling samples chance, opponent actions, and the update player’s own actions. So one recursive call follows a single terminal trajectory instead of branching at the update player’s information sets. This makes each iteration cheaper, but the update has to correct for the fact that most actions were not sampled.

Let $\sigma(I)$ be the regret-matching strategy at information set I . For the update player, actions are sampled from an exploration policy

$$q(I, a) = (1 - \epsilon)\sigma(I, a) + \epsilon \frac{1}{|A(I)|}.$$

In the experiments I used $\epsilon = 0.6$, matching a common vanilla outcome-sampling choice. If action a^* is sampled at I , the sampled child value is corrected by

$$\hat{u}(a^*) = \frac{u(a^*)}{q(I, a^*)}, \quad \hat{u}(a) = 0 \quad \text{for } a \neq a^*.$$

The regret update then uses

$$\frac{\pi_{-i}(I)}{q(h)} \left(\hat{u}(a) - \sum_{b \in A(I)} \sigma(I, b) \hat{u}(b) \right),$$

where $q(h)$ is the probability of sampling the trajectory prefix leading to I . This is where importance sampling enters the algorithm: sampled events that were unlikely under q receive larger weight. The target is still the same counterfactual-regret update as CFR, but the estimate can have much higher variance.

1.6 Implementation

The code is self-contained and intentionally small. It follows the direct tabular style of my earlier CFR repository [1], but does not import that code. The Kuhn Poker game is represented as a dictionary of extensive-form nodes. Chance nodes store the six equally likely card deals, decision nodes store player, information set, and action list, and terminal nodes store the zero-sum utility vector.

All three training algorithms share the same basic interface:

```
train(iterations),    current_strategy(),    average_strategy().
```

The full-tree CFR class keeps the standard counterfactual reach-probability weights. The MCCFR classes keep the same regret tables but only traverse a sampled part of the tree during each update. I also implemented exact evaluation helpers for expected value, best response value, and exploitability. For exploitability, the behavioral strategy is converted to sequence-form realization weights and then evaluated against exact best responses.

The main source files are organized as follows:

- `kuhn_poker.py`: explicit Kuhn Poker game tree used for sanity checks;

- `leduc_poker.py`: generated fixed-limit Leduc Poker game tree;
- `regret_matching.py`: positive-regret normalization with a uniform fallback;
- `cfr.py`: full-tree CFR baseline;
- `external_sampling_mccfr.py`: external-sampling MCCFR;
- `outcome_sampling_mccfr.py`: outcome-sampling MCCFR with epsilon exploration and importance weights;
- `util.py`: expected value, best response, exploitability, and strategy formatting helpers.

Exploitability is computed exactly rather than estimated from rollouts. For each player, the best-response recursion carries distributions over histories weighted only by chance and opponent reach. When the best-response player acts, histories are grouped by information set, so one action is chosen for the whole information set rather than separately at each hidden state. The reported exploitability is

$$\frac{1}{2}(\text{BR}_0(\bar{\sigma}_1) + \text{BR}_1(\bar{\sigma}_0)),$$

which is NashConv divided by two for a two-player zero-sum game.

I also count how many tree nodes are visited during regret updates. This counter is not needed by the algorithm, but it helped me compare full-tree CFR and MCCFR more directly. Full CFR visits the whole tree on each update. External sampling visits only a sampled part of the tree, although it still branches over the update player’s own actions.

1.7 Experiments

The main experiment compares full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR on Leduc Poker. I ran full-tree CFR for 500 iterations, external sampling for 100,000 iterations, and outcome sampling for 300,000 iterations. The MCCFR numbers below are averaged over three random seeds.

Algorithm	Iterations	Value	Exploitability	Regret-update nodes
CFR	500	−0.091170	0.030586	9.45×10^6
ES-MCCFR	100,000	−0.088230	0.041565	4.23×10^6
OS-MCCFR	300,000	−0.093453	0.237494	4.13×10^6

All three methods reduce exploitability from the uniform-strategy baseline, but they behave differently. CFR has the smoothest curve because every update is exact for the current profile. External sampling is noisier, but still stable because the update player branches over all actions at visited information sets. Outcome sampling is the noisiest in my runs: it uses importance weighting to learn from one sampled trajectory, so the variance shows up clearly in the curve.

The node-count comparison is where MCCFR looked most useful. In this implementation, full CFR visits about 18,902 tree nodes per iteration, external sampling visits about 42.6 nodes per iteration, and outcome sampling visits about 13.7 nodes per iteration. Outcome sampling is the cheapest per iteration, but in this experiment it needed many more samples to overcome variance.

I also plot the same curves against wall-clock time. This is a useful secondary view because node visits do not include Python overhead, sampling overhead, or the cost of periodically evaluating exploitability. In this small Leduc experiment, the wall-clock view does not show a clear speed advantage for MCCFR. The sampled algorithms visit far fewer regret update nodes, but they need many more iterations, and their sampling overhead and variance are visible in elapsed time. So the node-count plot is the cleaner algorithmic comparison, while the wall-clock plot is a more honest measurement of this particular implementation.

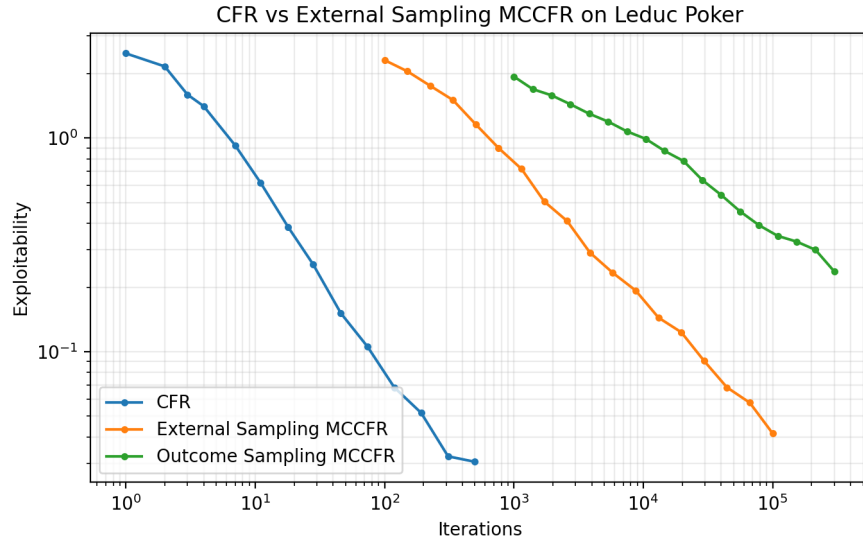


Figure 1 Exploitability of full-tree CFR, external-sampling MCCFR, and outcome-sampling MCCFR on Leduc Poker.

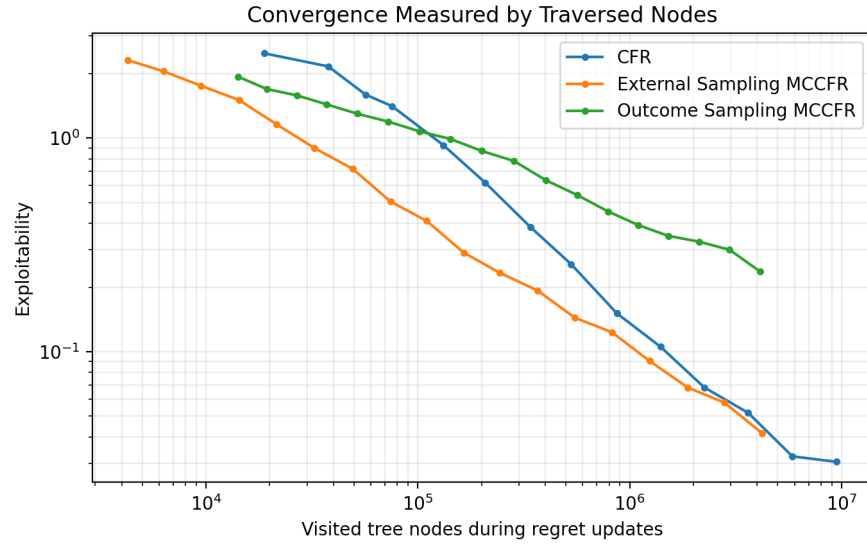


Figure 2 The same convergence curves measured by the number of tree nodes visited during regret updates.

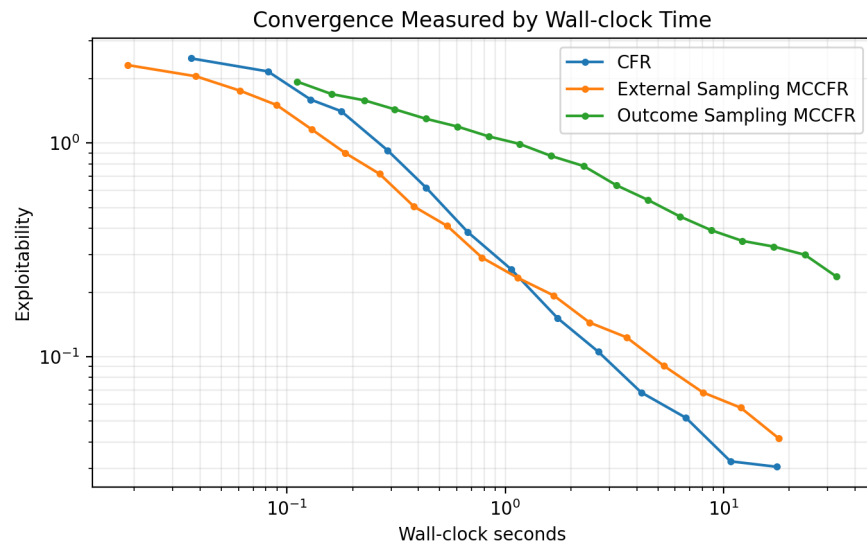


Figure 3 The same convergence curves measured by elapsed wall-clock seconds.

The external-sampling learned strategies also looked reasonable for Leduc Poker. This was a useful sanity check because a small exploitability number alone does not always make implementation mistakes easy to spot.

1.8 Learned Strategies

The learned average strategies also give a more concrete check on what the algorithm learned. The following table shows selected information sets from one run. The exact numbers are not unique, and the Leduc information-set labels are longer because they include private rank, public rank, betting round, and public betting history. Still, the broad pattern makes sense: weak unpaired hands are cautious, paired hands become aggressive, and facing a bet with a weak hand often leads to folding.

Card/public/history	Actions	First	Second	Third
<i>J</i> , no public, round 1 start	check/bet	0.927	0.073	–
<i>J</i> , no public, facing check-bet	fold/call/raise	0.927	0.054	0.020
<i>J</i> , public <i>J</i> , round 2 start	check/bet	0.341	0.659	–
<i>J</i> , public <i>J</i> , high-pot round 2 start	check/bet	0.018	0.982	–

The most useful part of this table is not the exact decimal values, but the pattern. The same private *J* is weak before the public card is revealed, but becomes a strong paired hand when the public card is also *J*. The learned strategy responds to that change by betting much more often in the paired states.

1.9 Discussion and Bibliography

Viewed as a Monte Carlo project, MCCFR is an example of replacing an exact expectation with a sampled estimator in a sequential decision problem. Full-tree CFR computes counterfactual values by traversing the whole game tree. MCCFR asks how much of that computation can be replaced by sampled paths while still producing useful regret updates. The experiments made the usual Monte Carlo tradeoff visible: sampling reduces the amount of the tree touched by each update, but it introduces variance, and importance weighting in outcome sampling can make that variance large.

External sampling was the more stable estimator in these experiments because it still compares all of the update player’s actions at visited information sets. Outcome sampling is the more aggressive Monte Carlo version: it samples a single trajectory and uses importance weights to correct for the sampling distribution. That makes each update very cheap in terms of tree nodes, but it also made the convergence curve much noisier.

The wall-clock plot is a useful reminder that a Monte Carlo method should be judged against a computational budget, not just by iteration count. In this small Leduc game and pure Python implementation, the lower traversal cost of MCCFR did not automatically become a wall-clock speedup because the algorithms needed many more samples and paid sampling overhead. I would expect the sampling advantage to matter more as the game tree grows larger, where full CFR’s complete traversal becomes the dominant cost. Natural next steps would be variance-reduction ideas for outcome sampling, larger Leduc variants, and abstraction methods such as bucketing.

References

- [1] Seongwoo Hur. Study and implementation of regret matching and counterfactual regret minimization algorithms. GitHub repository, 2026. <https://github.com/swhur1214/cfr>.
- [2] *Kuhn poker*. Wikipedia. https://en.wikipedia.org/wiki/Kuhn_poker. Accessed March 7, 2026.
- [3] Finnegan Southey, Michael P. Bowling, Bryce Larson, Carmelo Piccione, Neil Burch, Darse Billings, and Chris Rayner. Bayes’ bluff: Opponent modelling in poker. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, 2005. <https://arxiv.org/abs/1207.1411>.
- [4] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *Advances in Neural Information Processing Systems 20*, 2007. <https://papers.nips.cc/paper/3306-regret-minimization-in-games-with-incomplete-information>.
- [5] Marc Lanctot, Kevin Waugh, Martin Zinkevich, and Michael Bowling. Monte Carlo sampling for regret minimization in extensive games. In *Advances in Neural Information Processing Systems 22*, 2009. <https://papers.nips.cc/paper/3713-monte-carlo-sampling-for-regret-minimization-in-extensive-games>.
- [6] Gabriele Farina, Christian Kroer, and Tuomas Sandholm. Regret circuits: Composability of regret minimizers. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- [7] Gabriele Farina. *Computational Game Solving (CMU 15-888), Lectures 1–5*. 2021.
- [8] Harold W. Kuhn. A simplified two-person poker. In *Contributions to the Theory of Games*, 1950.